Bash : `set -euo pipefail`

Par défaut, un script Bash est **tolérant** : une commande qui échoue n'interrompt rien, une variable oubliée vaut une chaîne vide, un pipe masque les erreurs. Pour un script de déploiement, c'est dangereux : l'étape suivante s'exécute alors sur un état à moitié construit. La ligne `set -euo pipefail` rend le script **strict** : il s'arrête à la première erreur. Elle combine trois options, qui couvrent trois façons différentes de rater sans bruit.

> Les scripts cités vivent dans le dépôt Angular (`scripts/` , `update_version.sh` , `learning/build.sh`).

`-e` (errexit) : s'arrêter sur une commande en échec

Dès qu'une commande renvoie un code de sortie différent de 0, le script s'arrête avec ce code.

```
cp fichier.txt /inexistant/   # échoue -> le script s'arrête ici
echo "jamais exécuté"
```

Les commandes **testées** ne déclenchent pas l'arrêt : celles placées dans un `if` , un `while` , ou suivies de `&&` / `||` . C'est voulu (on a prévu le cas d'échec), mais c'est aussi la source des comportements subtils de `-e` .

`-u` (nounset) : refuser une variable non définie

Sans `-u` , une variable inconnue est remplacée par du vide, sans erreur :

```
rm -rf "$DOSSIER/"   # si $DOSSIER est vide : rm -rf "/"
```

Avec `-u` , le script s'arrête en signalant la variable fautive. Pour tolérer volontairement une variable absente, on donne une valeur par défaut avec `${VAR:-défaut}` (ou `${VAR:-}` pour « vide »).

`-o pipefail` : un pipe échoue si un maillon échoue

Par défaut, le code de sortie d'un pipe est celui de la **dernière** commande seulement.

```
cat fichier_inexistant | sort   # sans pipefail : code 0 (sort a réussi)
                                # avec pipefail : code 1
```

Avec `pipefail` , le pipe renvoie le code du dernier maillon en échec, et `-e` peut alors s'appliquer. Les trois options se complètent : `-e` ne servirait à rien sur un pipe sans `pipefail` .

Option	Piège évité	Sans l'option
`-e`	commande en échec ignorée	le script continue sur un état invalide
`-u`	faute de frappe ou variable oubliée	la variable vaut `""` en silence

Option	Piège évité	Sans l'option
`-o pipefail`	erreur d'un maillon avalée par le suivant	le pipe « réussit » si le dernier maillon réussit

Tolérer un échec voulu

Quand un échec est normal, on le déclare explicitement plutôt que de retirer la sécurité :

Besoin	Écriture
Ignorer l'échec d'une commande	`commande || true`
Tester sans arrêter le script	`if ! commande; then ...; fi`
Valeur par défaut d'une variable	`${VAR:-défaut}`
Nettoyer à la sortie, succès ou échec	`trap cleanup EXIT`

Cas classique : `grep` renvoie 1 quand il ne trouve rien, ce qui est une réponse, pas une panne. Avec `set -e`, on l'écrit `grep ... || true`, ou on le met dans un `if`.

Limites

`set -euo pipefail` n'est pas un filet de sécurité total :

- `-e` est ignoré dans les contextes testés (`if`, `&&`, `||`), y compris à l'intérieur d'une fonction appelée depuis un `if`.
- Un échec dans une substitution passée en argument (`echo "$(cmd_qui_echoue)"`) n'est pas détecté, seul le code de `echo` compte.
- Ces options protègent contre les erreurs **silencieuses**, pas contre les erreurs de logique : elles ne remplacent ni les vérifications explicites ni les tests.

Dans DataShare

> Le projet de ce dépôt est l'application Angular *Olympic Games*, déployée avec Helm.

Les cinq scripts Bash du dépôt commencent par `set -euo pipefail` : scripts/helm-deploy.sh:10, scripts/test-ghcr-image.sh:7, scripts/test-k8s-deploy.sh:21, update_version.sh:5 et learning/build.sh:9. Le hook `.githooks/pre-commit` fait de même.

Les trois options y sont utiles :

- `-u` : dans `helm-deploy.sh`, les arguments sont lus avec `ACTION=${1:-}` et `ENVIRONMENT=${2:-}`. Le `:-` déclare que l'absence d'argument est un cas prévu, traité ensuite par un `case` qui affiche l'usage. Le tableau `ACTION_ARGS` peut être vide pour `upgrade` ; l'écriture `${ACTION_ARGS[@]+"${ACTION_ARGS[@]}"}` évite l'erreur « variable non définie » que `-u` provoque sur un tableau vide avec les anciens Bash.
- `-e` : si `helm install` échoue, le script s'arrête et n'affiche pas « terminée. Suivre le démarrage… ».
- `|| true` **et** `trap` : `test-ghcr-image.sh` et `test-k8s-deploy.sh` définissent une fonction `cleanup` branchée par `trap cleanup EXIT`. Dedans, `docker rm -f ... || true` et `kubectl delete namespace`

`... || true` tolèrent un échec : le nettoyage ne doit pas masquer l'erreur d'origine ni s'interrompre. Dans `test-k8s-deploy.sh`, `curl ... || true` sert la fonction `http_code`, qui doit afficher `000` quand rien ne répond au lieu de faire tomber le script.

- `-o pipefail` : `test-k8s-deploy.sh` filtre la sortie de `helm install ... | grep -E ...`. Grâce à `pipefail`, un `helm install` en échec fait échouer la ligne même si `grep` réussit.

À retenir

1. `set -euo pipefail` = `-e` (stop sur échec) + `-u` (variable non définie = erreur) + `-o pipefail` (un pipe échoue si un maillon échoue). Chaque option bouche un trou de silence différent.
2. Sans `pipefail`, `-e` ne voit pas l'erreur d'un maillon de pipe : les trois options se complètent.
3. Un échec attendu se déclare (`|| true`, `if !`, `${VAR:-}`) au lieu de retirer la sécurité.
4. Ce n'est pas un filet total : `-e` est ignoré dans les contextes testés (`if`, `&&`, `||`).
5. Dans le projet, les cinq scripts l'utilisent ; `trap cleanup EXIT` garantit le nettoyage même quand `-e` interrompt le script.